

Домашний Шеф — Руководство по Внедрению



Содержание

1. [Быстрый старт](#)
 2. [Создание Product Master](#)
 3. [Интеграция Inventory](#)
 4. [Receipt Scanning Workflow](#)
 5. [Menu Generation Logic](#)
 6. [API Endpoints](#)
 7. [Frontend Components](#)
 8. [Database Setup](#)
-



Быстрый старт

Этап 1: Подготовка базовых данных

```
// 1. Создаем категории
const categories = {
  ML: { name: 'Молочные', icon: '🥛' },
  VG: { name: 'Овощи', icon: '🥬' },
  MT: { name: 'Мясо & Рыба', icon: '🥩' },
  PA: { name: 'Панир & Основы', icon: '🍲' }
};
```

```
// 2. Создаем первый продукт
const slivki = {
  productId: 'ML-CR-001',
  russianName: 'Сливки 33%',
  englishName: 'Cream 33%',
  originalName: 'Slagroom',
  category: 'ML',
  subcategory: 'CR',
  baseUnit: 'ml',
```

```
    storageType: 'Refrigerated',
    shelfLife: 10,
    openShelfLife: 3,
    minimumStock: 500,
    defaultCurrency: 'EUR',
    averagePrice: 2.49
  };
```

```
// 3. Сохраняем в БД
db.products.insert(slivki);
```

Создание Product Master

Шаг 1: Инициализация базы

```
class ProductMasterSystem {
  constructor(database) {
    this.db = database;
    this.products = new Map();
    this.categories = new Map();
  }

  // Создать категорию
  createCategory(code, name, icon) {
    const category = { code, name, icon };
    this.categories.set(code, category);
    return category;
  }

  // Добавить продукт
  addProduct(productData) {
    // Валидация Product ID
    if (!this.validateProductId(productData.productId)) {
      throw new Error('Invalid Product ID format');
    }

    // Проверка дублей
    if (this.products.has(productData.productId)) {
      throw new Error('Product ID already exists');
    }

    // Добавить временные метки
    productData.createdAt = new Date();
  }
}
```

```

    productData.updatedAt = new Date();
    productData.active = true;

    this.products.set(productData.productId, productData);
    return productData;
}

// Валидация Product ID формата
validateProductId(id) {
    const pattern = /^[A-Z]{2}-[A-Z]{2}-\d{3}$/;
    return pattern.test(id);
}

// Получить все продукты категории
getProductsByCategory(categoryCode) {
    return Array.from(this.products.values())
        .filter(p => p.category === categoryCode);
}

// Обновить Product (но НЕ Product ID)
updateProduct(productId, updates) {
    if (updates.productId && updates.productId !== productId) {
        throw new Error('Product ID is immutable');
    }

    const product = this.products.get(productId);
    if (!product) throw new Error('Product not found');

    const updated = {
        ...product,
        ...updates,
        updatedAt: new Date(),
        productId // гарантировать неизменность ID
    };

    this.products.set(productId, updated);
    return updated;
}

// Поиск продукта
searchProducts(query) {
    return Array.from(this.products.values()).filter(p =>
        p.russianName.includes(query) ||
        p.englishName.includes(query) ||
        p.productId.includes(query)
    );
}

```

```

}

// Использование
const pms = new ProductMasterSystem(database);

// Создание категорий
pms.createCategory('ML', 'Молочные', '🥛');
pms.createCategory('VG', 'Овощи', '🥬');

// Добавление продуктов
pms.addProduct({
  productId: 'ML-CR-001',
  russianName: 'Сливки 33%',
  englishName: 'Cream 33%',
  originalName: 'Slagroom',
  category: 'ML',
  subcategory: 'CR',
  baseUnit: 'ml',
  storageType: 'Refrigerated',
  shelfLife: 10,
  openShelfLife: 3,
  minimumStock: 500,
  defaultCurrency: 'EUR',
  averagePrice: 2.49
});

```

Шаг 2: Исходные данные (Seed Data)

```

// seedProducts.js - инициализация базовых продуктов

const SEED_PRODUCTS = [
  // Молочные
  {
    productId: 'ML-CR-001',
    russianName: 'Сливки 33%',
    englishName: 'Cream 33%',
    originalName: 'Slagroom',
    category: 'ML',
    subcategory: 'CR',
    baseUnit: 'ml',
    storageType: 'Refrigerated',
    shelfLife: 10,
    openShelfLife: 3,
    minimumStock: 500,
    defaultCurrency: 'EUR',
    averagePrice: 2.49,
  },

```

```
    supplier: 'Albert Heijn'
  },
  {
    productId: 'ML-CH-001',
    russianName: 'Сыр Пармезан',
    englishName: 'Parmesan Cheese',
    originalName: 'Parmezaanse Kaas',
    category: 'ML',
    subcategory: 'CH',
    baseUnit: 'g',
    storageType: 'Refrigerated',
    shelfLife: 180,
    openShelfLife: 30,
    minimumStock: 100,
    defaultCurrency: 'EUR',
    averagePrice: 14.99
  },
```

```
// Овощи
```

```
{
  productId: 'VG-TM-001',
  russianName: 'Томаты',
  englishName: 'Tomatoes',
  originalName: 'Tomaten',
  category: 'VG',
  subcategory: 'TM',
  baseUnit: 'kg',
  storageType: 'Room Temperature',
  shelfLife: 7,
  openShelfLife: 0,
  minimumStock: 1000,
  defaultCurrency: 'EUR',
  averagePrice: 3.99
},
```

```
{
  productId: 'VG-PP-001',
  russianName: 'Болгарский перец',
  englishName: 'Bell Pepper',
  originalName: 'Paprika',
  category: 'VG',
  subcategory: 'PP',
  baseUnit: 'kg',
  storageType: 'Refrigerated',
  shelfLife: 14,
  openShelfLife: 0,
  minimumStock: 500,
  defaultCurrency: 'EUR',
```

```

        averagePrice: 4.99
    },

    // Мясо
    {
        productId: 'MT-CH-001',
        russianName: 'Куриное филе',
        englishName: 'Chicken Fillet',
        originalName: 'Kipfilet',
        category: 'MT',
        subcategory: 'CH',
        baseUnit: 'kg',
        storageType: 'Frozen',
        shelfLife: 180,
        openShelfLife: 2,
        minimumStock: 500,
        defaultCurrency: 'EUR',
        averagePrice: 12.99
    },

    // Основы
    {
        productId: 'PA-OL-001',
        russianName: 'Оливковое масло',
        englishName: 'Olive Oil',
        originalName: 'Olijfolie',
        category: 'PA',
        subcategory: 'OL',
        baseUnit: 'ml',
        storageType: 'Pantry',
        shelfLife: 730,
        openShelfLife: 365,
        minimumStock: 250,
        defaultCurrency: 'EUR',
        averagePrice: 8.99
    }
];

// Функция для импорта
async function seedDatabase() {
    for (const product of SEED_PRODUCTS) {
        await db.products.insert(product);
    }
    console.log('✅ Database seeded with products');
}

```

Создание связи между Product Master и Inventory

```
class InventorySystem {
  constructor(productMaster, database) {
    this.pms = productMaster;
    this.db = database;
    this.inventory = new Map();
  }

  // Добавить товар в инвентарь
  addInventoryItem(inventoryData) {
    // Проверить, что Product существует
    if (!this.pms.products.has(inventoryData.productId)) {
      throw new Error(`Product ${inventoryData.productId} not found`);
    }

    const product = this.pms.products.get(inventoryData.productId);

    // Валидировать единицы измерения
    if (inventoryData.unit !== product.baseUnit) {
      console.warn(
        `Unit mismatch: inventory ${inventoryData.unit}, product ${product.baseUnit}`
      );
    }

    // Вычислить срок годности
    const purchaseDate = new Date(inventoryData.purchaseDate);
    const expirationDate = new Date(purchaseDate);
    expirationDate.setDate(
      expirationDate.getDate() + product.shelfLife
    );
    inventoryData.expirationDate = expirationDate.toISOString().split('T')[0];

    // Установить начальный статус
    inventoryData.status = 'Fresh';
    inventoryData.createdAt = new Date();

    this.inventory.set(inventoryData.inventoryId, inventoryData);
    return inventoryData;
  }

  // Получить все товары по продукту
  getByProduct(productId) {
    return Array.from(this.inventory.values())
  }
}
```

```

        .filter(item => item.productId === productId);
    }

    // Получить товары по локации
    getByLocation(location) {
        return Array.from(this.inventory.values())
            .filter(item => item.location === location);
    }

    // Обновить статус товара
    updateStatus(inventoryId, newStatus) {
        const item = this.inventory.get(inventoryId);
        if (!item) throw new Error('Inventory item not found');

        item.status = newStatus;
        if (newStatus === 'Opened' && !item.openDate) {
            item.openDate = new Date().toISOString().split('T')[0];
        }

        return item;
    }

    // Проверить товары, истекающие сроки
    getExpiringItems(daysThreshold = 2) {
        const today = new Date();
        const threshold = new Date(today);
        threshold.setDate(threshold.getDate() + daysThreshold);

        return Array.from(this.inventory.values()).filter(item => {
            if (item.status === 'Expired') return false;
            const expDate = new Date(item.expirationDate);
            return expDate <= threshold && expDate > today;
        });
    }

    // Получить истекшие товары
    getExpiredItems() {
        const today = new Date();
        return Array.from(this.inventory.values()).filter(item => {
            const expDate = new Date(item.expirationDate);
            return expDate < today && item.status !== 'Expired';
        });
    }

    // Использованный товар (удалить из инвентаря или отметить)
    consumeItem(inventoryId, quantity) {
        const item = this.inventory.get(inventoryId);
    }

```

```

        if (!item) throw new Error('Inventory item not found');

        item.quantity -= quantity;
        if (item.quantity <= 0) {
            item.status = 'Used';
            this.inventory.delete(inventoryId);
        }

        return item;
    }

    // Получить краткую статистику инвентаря
    getStatistics() {
        const items = Array.from(this.inventory.values());
        const totalItems = items.length;
        const fresh = items.filter(i => i.status === 'Fresh').length;
        const opened = items.filter(i => i.status === 'Opened').length;
        const expiring = this.getExpiringItems().length;
        const expired = items.filter(i => i.status === 'Expired').length;

        return {
            totalItems,
            fresh,
            opened,
            expiring,
            expired
        };
    }
}

// Использование
const inventory = new InventorySystem(pms, database);

// Добавить товар в инвентарь
inventory.addInventoryItem({
    inventoryId: 'INV-001',
    productId: 'ML-CR-001',
    quantity: 500,
    unit: 'ml',
    location: 'Refrigerator',
    purchaseDate: '2024-01-15',
    costPerUnit: 2.49,
    currency: 'EUR'
});

// Получить товары, которые скоро истекают
const expiringItems = inventory.getExpiringItems(2);

```

```
console.log('Items expiring in 2 days:', expiringItems);
```

Receipt Scanning Workflow

Сканирование чека → Добавление в инвентарь

```
class ReceiptSystem {
  constructor(productMaster, inventorySystem, translationAPI) {
    this.pms = productMaster;
    this.inventory = inventorySystem;
    this.translator = translationAPI;
    this.receipts = new Map();
  }

  // Основной workflow: обработка чека
  async processReceipt(receiptImage) {
    // 1. Распознавание текста (OCR)
    const ocrText = await this.performOCR(receiptImage);
    console.log('📄 OCR Recognized text');

    // 2. Парсинг товаров
    const items = this.parseReceiptItems(ocrText);
    console.log(`📦 Parsed ${items.length} items`);

    // 3. Перевод на русский
    const translatedItems = await this.translateItems(items);
    console.log('🌐 Translated to Russian');

    // 4. Поиск совпадений в Product Master
    const matchedItems = this.matchProducts(translatedItems);
    console.log(`✅ Matched ${matchedItems.filter(m => m.matched).length} items`);

    // 5. Создание записей в Inventory
    const inventoryItems = await this.createInventoryItems(matchedItems);
    console.log(`🇷🇺 Added ${inventoryItems.length} to inventory`);

    // 6. Сохранить чек
    const receipt = {
      receiptId: this.generateReceiptId(),
      items: matchedItems,
      date: new Date(),
      ocr_raw: ocrText
    };
  }
}
```

```

    this.receipts.set(receipt.receiptId, receipt);

    return receipt;
}

// Шаг 1: OCR
async performOCR(imageData) {
    // Использование Google Vision API, Azure, или Tesseract
    // Для примера - mock
    return `
        Slagroom - 1 x 2.49 EUR
        Tomaten - 2kg x 3.99 EUR
        Kipfilet - 1kg x 12.99 EUR
    `;
}

// Шаг 2: Парсинг чека
parseReceiptItems(text) {
    const lines = text.split('\n');
    const items = [];

    for (const line of lines) {
        // Простой парсинг: "Name - quantity x price"
        const match = line.match(/(.+?)\s*-\s*([\d.]+\s*x?\s*([\d.]+\s*(\w+)))/);
        if (match) {
            items.push({
                originalName: match[1].trim(),
                quantity: parseFloat(match[2]),
                unitPrice: parseFloat(match[3]),
                currency: match[4]
            });
        }
    }

    return items;
}

// Шаг 3: Перевод
async translateItems(items) {
    // Использование встроенного словаря + Google Translate API
    const DUTCH_DICTIONARY = {
        'slagroom': 'Сливки',
        'tomaten': 'Томаты',
        'kipfilet': 'Куриное филе',
        'kaas': 'Сыр',
        'melk': 'Молоко',
        'olijfolie': 'Оливковое масло'
    };

```

```

    });

    return items.map(item => ({
      ...item,
      russianName:
        DUTCH_DICTIONARY[item.originalName.toLowerCase()] ||
        item.originalName // fallback
    }));
  }

  // Шаг 4: Поиск совпадений
  matchProducts(items) {
    return items.map(item => {
      // Поиск по названию
      const matches = Array.from(this.pms.products.values())
        .filter(p =>
          p.originalName.toLowerCase() === item.originalName.toLowerCase() ||
          p.russianName.toLowerCase() === item.russianName.toLowerCase()
        );

      if (matches.length > 0) {
        return {
          ...item,
          matched: true,
          productId: matches[0].productId,
          confidence: 0.95
        };
      }

      return {
        ...item,
        matched: false,
        confidence: 0
      };
    });
  }

  // Шаг 5: Добавить в инвентарь
  async createInventoryItems(matchedItems) {
    const created = [];

    for (const item of matchedItems) {
      if (!item.matched) continue;

      const product = this.pms.products.get(item.productId);
      const inventoryId = `INV-${Date.now()}-${item.productId}`;

```

```

    try {
      const inventoryItem = this.inventory.addInventoryItem({
        inventoryId,
        productId: item.productId,
        quantity: item.quantity * this.getBaseUnitMultiplier(product.baseUnit),
        unit: product.baseUnit,
        location: 'Pantry', // default
        purchaseDate: new Date().toISOString().split('T')[0],
        costPerUnit: item.unitPrice,
        currency: item.currency
      });

      created.push(inventoryItem);
    } catch (error) {
      console.error(`Failed to add ${item.productId}:`, error);
    }
  }

  return created;
}

// Вспомогательная функция для конвертации единиц
getBaseUnitMultiplier(unit) {
  const multipliers = {
    'g': 1,
    'kg': 1000,
    'ml': 1,
    'l': 1000,
    'piece': 1
  };
  return multipliers[unit] || 1;
}

generateReceiptId() {
  const date = new Date();
  const dateStr = date.toISOString().split('T')[0].replace(/-/g, '');
  const random = Math.random().toString(36).substr(2, 3);
  return `REC-${dateStr}-${random}`;
}

// Использование
const receiptSystem = new ReceiptSystem(pms, inventory);

// Обработка чека (переданного как File или base64)
const receipt = await receiptSystem.processReceipt(receiptImage);
console.log('Receipt processed:', receipt);

```

Menu Generation Logic

Автоматическое создание меню из доступных продуктов

```
class MenuSystem {
  constructor(productMaster, inventorySystem, recipesDB) {
    this.pms = productMaster;
    this.inventory = inventorySystem;
    this.recipes = recipesDB;
  }

  // Основная функция: генерировать меню
  generateMenu(preferences = {}) {
    // 1. Получить текущий инвентарь
    const items = this.inventory.getAll();

    // 2. Приоритизировать товары
    const prioritized = this.prioritizeItems(items);

    // 3. Найти рецепты, которые можно готовить
    const availableRecipes = this.findAvailableRecipes(prioritized);

    // 4. Выбрать блюда для меню
    const selectedRecipes = this.selectRecipes(availableRecipes, preferences);

    // 5. Создать меню
    const menu = {
      menuId: `MNU-${new Date().toISOString().split('T')[0]}`,
      meals: this.organizeMeals(selectedRecipes),
      recipes: selectedRecipes,
      reservedInventory: this.reserveItems(selectedRecipes),
      shoppingNeeded: this.calculateShopping(selectedRecipes)
    };

    return menu;
  }

  // Шаг 1: Приоритизация товаров
  prioritizeItems(items) {
    const today = new Date();

    return items.map(item => {
      let priority = 100; // базовый приоритет
```

```

    // Товары, истекающие сроки (высший приоритет)
    if (item.status === 'Expiring Soon') priority = 1;
    if (item.status === 'Expired') priority = 0;

    // Открытые товары
    if (item.status === 'Opened') priority = Math.min(priority, 10);

    // Старые товары (долго лежат)
    const daysSincePurchase = Math.floor(
      (today - new Date(item.purchaseDate)) / (1000 * 60 * 60 * 24)
    );
    if (daysSincePurchase > 5) priority = Math.min(priority, 20);

    return { ...item, priority };
  }).sort((a, b) => a.priority - b.priority);
}

// Шаг 2: Поиск доступных рецептов
findAvailableRecipes(prioritizedItems) {
  const available = [];

  for (const recipe of this.recipes.getAll()) {
    const canMake = this.canMakeRecipe(recipe, prioritizedItems);
    if (canMake) {
      available.push({
        recipe,
        usesProducts: this.getUsedProducts(recipe, prioritizedItems),
        score: this.calculateRecipeScore(recipe, prioritizedItems)
      });
    }
  }
}

// Сортировать по очкам (сколько приоритетных товаров используется)
return available.sort((a, b) => b.score - a.score);
}

// Проверить, можно ли готовить рецепт
canMakeRecipe(recipe, items) {
  for (const ingredient of recipe.ingredients) {
    const hasIngredient = items.some(
      item => item.productId === ingredient.productId &&
        item.quantity >= ingredient.quantity
    );
    if (!hasIngredient) return false;
  }
  return true;
}

```

```

}

// Получить используемые товары
getUsedProducts(recipe, items) {
  return recipe.ingredients.map(ing => {
    const item = items.find(i => i.productId === ing.productId);
    return { ...item, neededQuantity: ing.quantity };
  });
}

// Вычислить очки рецепта (сколько приоритетных товаров используется)
calculateRecipeScore(recipe, items) {
  let score = 0;
  for (const ingredient of recipe.ingredients) {
    const item = items.find(i => i.productId === ingredient.productId);
    if (item) score += (100 - item.priority); // чем выше приоритет, тем больше
  }
  return score;
}

// Шаг 3: Выбор рецептов для меню
selectRecipes(availableRecipes, preferences = {}) {
  const selected = [];
  const mealsPerDay = preferences.mealsPerDay || 3;

  // Выбрать топ рецепты по очкам
  for (let i = 0; i < mealsPerDay && i < availableRecipes.length; i++) {
    selected.push(availableRecipes[i].recipe);
  }

  return selected;
}

// Шаг 4: Организовать блюда по времени дня
organizeMeals(recipes) {
  const meals = [];
  const mealTypes = ['breakfast', 'lunch', 'dinner'];

  for (let i = 0; i < recipes.length && i < mealTypes.length; i++) {
    meals.push({
      mealType: mealTypes[i],
      recipes: [recipes[i].recipeId]
    });
  }

  return meals;
}

```

```

// Шаг 5: Зарезервировать товары
reserveItems(recipes) {
  const reserved = [];

  for (const recipe of recipes) {
    for (const ingredient of recipe.ingredients) {
      // Найти товар в инвентаре
      const items = this.inventory.getByProduct(ingredient.productId);
      let needed = ingredient.quantity;

      for (const item of items) {
        if (needed <= 0) break;

        const toUse = Math.min(item.quantity, needed);
        reserved.push({
          inventoryId: item.inventoryId,
          quantity: toUse,
          unit: item.unit,
          recipe: recipe.recipeId
        });

        needed -= toUse;
      }
    }
  }

  return reserved;
}

// Шаг 6: Рассчитать, что нужно купить
calculateShopping(recipes) {
  const needed = new Map();

  for (const recipe of recipes) {
    for (const ingredient of recipe.ingredients) {
      const current = this.inventory.getByProduct(ingredient.productId);
      const available = current.reduce((sum, item) => sum + item.quantity, 0);

      if (available < ingredient.quantity) {
        const productNeeded = ingredient.quantity - available;
        const key = ingredient.productId;

        if (!needed.has(key)) {
          needed.set(key, {
            productId: ingredient.productId,
            quantity: productNeeded,

```

```

        unit: this.pms.products.get(ingredient.productId).baseUnit
    });
} else {
    const item = needed.get(key);
    item.quantity += productNeeded;
}
}
}
}

return Array.from(needed.values());
}
}

// Использование
const menuSystem = new MenuSystem(pms, inventory, recipesDatabase);

const menu = menuSystem.generateMenu({
    mealsPerDay: 3,
    cuisinePreference: 'Italian'
});

console.log('Generated menu:', menu);
console.log('Shopping needed:', menu.shoppingNeeded);

```

API Endpoints

REST API структура

```

// API.md - RESTful endpoints

// ===== PRODUCT MASTER =====

GET    /api/products
Return: Array of all products
Query: ?category=ML&skip=0&limit=20

GET    /api/products/:productId
Return: Single product data

POST   /api/products
Create: New product
Body: { russianName, englishName, category, ... }

```

Return: Created product

PATCH /api/products/:productId
Update: Product data (except productId)
Body: { averagePrice, minimumStock, ... }
Return: Updated product

DELETE /api/products/:productId
Soft delete: Set active=false
Return: Confirmation

GET /api/products/search/:query
Search: By name or product ID
Return: Array of matching products

// ===== INVENTORY =====

GET /api/inventory
Return: All inventory items
Query: ?location=Refrigerator&status=Fresh

GET /api/inventory/:inventoryId
Return: Single inventory item details

POST /api/inventory
Add: New inventory item
Body: { productId, quantity, location, ... }
Return: Created inventory item

PATCH /api/inventory/:inventoryId
Update: Quantity, location, status
Body: { quantity, location, status }
Return: Updated item

DELETE /api/inventory/:inventoryId
Remove: Item from inventory
Return: Confirmation

GET /api/inventory/product/:productId
Return: All items of a product

GET /api/inventory/expiring
Return: Items expiring in N days
Query: ?days=2

// ===== RECEIPTS =====

POST /api/receipts/scan
Upload: Receipt image for OCR processing
Body: FormData with image
Return: Processed receipt with matched items

GET /api/receipts
Return: All receipts
Query: ?from=2024-01-01&to=2024-01-31

GET /api/receipts/:receiptId
Return: Receipt details

POST /api/receipts/:receiptId/confirm
Confirm: Receipt and add to inventory
Body: { itemsToAdd: [...] }
Return: Created inventory items

// ===== RECIPES =====

GET /api/recipes
Return: All recipes
Query: ?difficulty=Easy&tags=quick

POST /api/recipes
Create: New recipe
Body: { name, ingredients: [{ productId, quantity, ... }], ... }
Return: Created recipe

// ===== MENU =====

POST /api/menu/generate
Generate: Today's menu
Query: ?meals=3&preferences=vegetarian
Return: Generated menu with recommendations

POST /api/menu/save
Save: Menu to history
Body: { meals, reservedInventory, shoppingNeeded }
Return: Saved menu ID

// ===== SHOPPING =====

GET /api/shopping/list
Return: Current shopping list (from menu + low stock)

POST /api/shopping/add
Add: Item to shopping list

```

    Body: { productId, quantity }
    Return: Updated list

POST    /api/shopping/check-off
    Check: Item as purchased
    Body: { productId, quantity, actualPrice }
    Return: Updated list

// ===== ANALYTICS =====

GET     /api/analytics/usage
    Usage: Statistics
    Return: Most used products, categories

GET     /api/analytics/costs
    Costs: By category and time
    Query: ?from=2024-01-01&to=2024-01-31
    Return: Cost breakdown

GET     /api/analytics/waste
    Waste: Products that expire often
    Return: Waste statistics

```

Пример использования API (JavaScript/Fetch)

```

// fetchProductMaster.js

class HomeChefAPI {
  constructor(baseUrl = 'http://localhost:3000/api') {
    this.baseUrl = baseUrl;
  }

  // Получить все продукты
  async getProducts(category = null) {
    let url = `${this.baseUrl}/products`;
    if (category) url += `?category=${category}`;

    const response = await fetch(url);
    return response.json();
  }

  // Поиск продукта
  async searchProducts(query) {
    const response = await fetch(
      `${this.baseUrl}/products/search/${encodeURIComponent(query)}`
    );
  }
}

```

```

    return response.json();
}

// Создать новый продукт
async createProduct(productData) {
    const response = await fetch(`${this.baseURL}/products`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(productData)
    });
    return response.json();
}

// Добавить товар в инвентарь
async addToInventory(inventoryData) {
    const response = await fetch(`${this.baseURL}/inventory`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(inventoryData)
    });
    return response.json();
}

// Сканировать чек
async scanReceipt(imageFile) {
    const formData = new FormData();
    formData.append('image', imageFile);

    const response = await fetch(`${this.baseURL}/receipts/scan`, {
        method: 'POST',
        body: formData
    });
    return response.json();
}

// Генерировать меню
async generateMenu(preferences = {}) {
    const response = await fetch(`${this.baseURL}/menu/generate`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(preferences)
    });
    return response.json();
}

// Получить товары, истекающие сроки
async getExpiringItems(days = 2) {

```

```

    const response = await fetch(
      `${this.baseUrl}/inventory/expiring?days=${days}`
    );
    return response.json();
  }
}

// Использование
const api = new HomeChefAPI('http://localhost:3000/api');

// Получить молочные продукты
const dairy = await api.getProducts('ML');
console.log('Dairy products:', dairy);

// Поиск сливок
const creams = await api.searchProducts('сливки');
console.log('Found creams:', creams);

// Добавить товар
const newItem = await api.addToInventory({
  productId: 'ML-CR-001',
  quantity: 500,
  unit: 'ml',
  location: 'Refrigerator',
  purchaseDate: '2024-01-15'
});
console.log('Added to inventory:', newItem);

// Сканировать чек
const receiptFile = document.querySelector('input[type="file"]').files[0];
const receipt = await api.scanReceipt(receiptFile);
console.log('Receipt processed:', receipt);

// Генерировать меню
const menu = await api.generateMenu({ mealsPerDay: 3 });
console.log('Menu generated:', menu);

```

Frontend Components

React компоненты системы

```
// components/ProductMasterViewer.jsx
```

```

import React, { useState, useEffect } from 'react';

function ProductMasterViewer() {
  const [products, setProducts] = useState([]);
  const [selectedCategory, setSelectedCategory] = useState('ML');
  const [searchQuery, setSearchQuery] = useState('');
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    loadProducts();
  }, [selectedCategory]);

  async function loadProducts() {
    setLoading(true);
    try {
      const response = await fetch(`/api/products?category=${selectedCategory}`);
      const data = await response.json();
      setProducts(data);
    } catch (error) {
      console.error('Failed to load products:', error);
    } finally {
      setLoading(false);
    }
  }

  async function handleSearch(e) {
    e.preventDefault();
    if (!searchQuery.trim()) {
      loadProducts();
      return;
    }

    setLoading(true);
    try {
      const response = await fetch(
        `/api/products/search/${encodeURIComponent(searchQuery)}`
      );
      const data = await response.json();
      setProducts(data);
    } catch (error) {
      console.error('Search failed:', error);
    } finally {
      setLoading(false);
    }
  }

  return (

```

```

<div className="p-6">
  <h1 className="text-3xl font-bold mb-6">Product Master</h1>

  {/* Search */}
  <form onSubmit={handleSearch} className="mb-6">
    <input
      type="text"
      placeholder="Search products..."
      value={searchQuery}
      onChange={(e) => setSearchQuery(e.target.value)}
      className="px-4 py-2 border rounded w-full"
    />
  </form>

  {/* Category Tabs */}
  <div className="flex gap-4 mb-6">
    {['ML', 'VG', 'MT', 'PA'].map(cat => (
      <button
        key={cat}
        onClick={() => setSelectedCategory(cat)}
        className={`px-4 py-2 rounded ${
          selectedCategory === cat
            ? 'bg-blue-500 text-white'
            : 'bg-gray-200'
        }}
      >
        {cat}
      </button>
    ))}
  </div>

  {/* Products Grid */}
  {loading ? (
    <div>Loading...</div>
  ) : (
    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
      {products.map(product => (
        <div key={product.productId} className="border rounded p-4">
          <h3 className="font-bold text-lg">{product.russianName}</h3>
          <p className="text-sm text-gray-600">{product.englishName}</p>
          <p className="text-xs text-blue-600 font-mono mt-2">
            {product.productId}
          </p>
          <div className="mt-4 text-sm">
            <p>Unit: {product.baseUnit}</p>
            <p>Storage: {product.storageType}</p>
            <p>Shelf Life: {product.shelfLife} days</p>
          </div>
        </div>
      ))}
    </div>
  )}

```

```

                <p>Price: {product.averagePrice} {product.defaultCurrency}</p>
            </div>
        </div>
    )}
</div>
})
</div>
);
}

```

```
export default ProductMasterViewer;
```

```
// components/InventoryManager.jsx
```

```
import React, { useState, useEffect } from 'react';
```

```
function InventoryManager() {
    const [inventory, setInventory] = useState([]);
    const [selectedLocation, setSelectedLocation] = useState('Refrigerator');
```

```
    useEffect(() => {
        loadInventory();
    }, [selectedLocation]);
```

```
    async function loadInventory() {
        const response = await fetch(
            `/api/inventory?location=${selectedLocation}`
        );
        const data = await response.json();
        setInventory(data);
    }
```

```
    async function updateStatus(inventoryId, newStatus) {
        const response = await fetch(`/api/inventory/${inventoryId}`, {
            method: 'PATCH',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ status: newStatus })
        });
        if (response.ok) loadInventory();
    }
```

```
    return (
        <div className="p-6">
            <h1 className="text-3xl font-bold mb-6">Inventory</h1>

            {/* Location Filter */}

```

```

<div className="flex gap-2 mb-6">
  {[ 'Refrigerator', 'Freezer', 'Pantry', 'Prep' ].map(loc => (
    <button
      key={loc}
      onClick={() => setSelectedLocation(loc)}
      className={`px-4 py-2 rounded ${
        selectedLocation === loc
          ? 'bg-green-500 text-white'
          : 'bg-gray-200'
      }}
    >
      {loc}
    </button>
  ))}
</div>

{/* Inventory Items */}
<div className="space-y-4">
  {inventory.map(item => (
    <div key={item.inventoryId} className="border rounded p-4 flex justify-
      <div>
        <h3 className="font-bold">{item.productId}</h3>
        <p className="text-sm text-gray-600">
          {item.quantity} {item.unit}
        </p>
        <p className="text-xs text-gray-500">
          Expires: {item.expirationDate}
        </p>
      </div>
      <select
        value={item.status}
        onChange={(e) => updateStatus(item.inventoryId, e.target.value)}
        className="px-3 py-1 border rounded"
      >
        <option>Fresh</option>
        <option>Opened</option>
        <option>Frozen</option>
        <option>Expiring Soon</option>
        <option>Expired</option>
      </select>
    </div>
  ))}
</div>
</div>
);
}

```

```
export default InventoryManager;
```

Database Setup

PostgreSQL Schema

```
-- products.sql

CREATE TABLE categories (
  code VARCHAR(2) PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  icon VARCHAR(10)
);

CREATE TABLE products (
  product_id VARCHAR(15) PRIMARY KEY,
  russian_name VARCHAR(255) NOT NULL,
  english_name VARCHAR(255),
  original_name VARCHAR(255),

  category_code VARCHAR(2) NOT NULL REFERENCES categories(code),
  subcategory_code VARCHAR(2) NOT NULL,

  base_unit VARCHAR(20) NOT NULL,
  storage_type VARCHAR(50),
  shelf_life_days INT,
  open_shelf_life_days INT,
  minimum_stock DECIMAL(10, 2),

  default_currency VARCHAR(3),
  average_price DECIMAL(10, 2),
  supplier VARCHAR(255),

  active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_products_category ON products(category_code);
CREATE INDEX idx_products_active ON products(active);

-- inventory.sql
```

```
CREATE TABLE inventory (  
  inventory_id VARCHAR(50) PRIMARY KEY,  
  product_id VARCHAR(15) NOT NULL REFERENCES products(product_id),  
  
  quantity DECIMAL(10, 3) NOT NULL,  
  unit VARCHAR(20) NOT NULL,  
  
  location VARCHAR(50) NOT NULL,  
  purchase_date DATE NOT NULL,  
  expiration_date DATE NOT NULL,  
  open_date DATE,  
  
  status VARCHAR(50) DEFAULT 'Fresh',  
  batch_id VARCHAR(50),  
  
  cost_per_unit DECIMAL(10, 2),  
  currency VARCHAR(3),  
  
  notes TEXT,  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_inventory_product ON inventory(product_id);  
CREATE INDEX idx_inventory_location ON inventory(location);  
CREATE INDEX idx_inventory_status ON inventory(status);  
CREATE INDEX idx_inventory_expiration ON inventory(expiration_date);
```



Документация по развертыванию

Docker Compose

```
# docker-compose.yml  
  
version: '3.8'  
  
services:  
  postgres:  
    image: postgres:15  
    environment:  
      POSTGRES_DB: homeChef  
      POSTGRES_USER: chef  
      POSTGRES_PASSWORD: securePassword123
```

```
ports:
  - "5432:5432"
volumes:
  - ./init.sql:/docker-entrypoint-initdb.d/init.sql
  - postgres_data:/var/lib/postgresql/data

api:
  build: ./backend
  environment:
    DATABASE_URL: postgresql://chef:securePassword123@postgres:5432/homeChef
    NODE_ENV: development
  ports:
    - "3000:3000"
  depends_on:
    - postgres

frontend:
  build: ./frontend
  ports:
    - "3001:3001"
  depends_on:
    - api

volumes:
  postgres_data:
```

Запуск:

```
docker-compose up -d
```

Эта документация обеспечивает полную базу для внедрения всей системы! 🚀